

Intel Arc Pro B70 + Qwen3.8-27B

vLLM XPU Replication Manual

Known-good Dell validation-bench configuration • September 3, 2026

Purpose: preserve the exact working configuration, troubleshooting history, launch commands, and benchmark procedure needed to retrieve and reproduce the successful Qwen3.8-27B GPTQ INT4 + BF16 MTP4 deployment on an Intel Arc Pro B70.

1. Final result

Metric	Measured result
Model	SergiiB/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16
Engine	vLLM XPU 0.27.2rc1.dev77+gac7509e2b.xpu
XPU kernels	vllm-xpu-kernels 0.1.12.3
Speculative decoding	MTP4, BF16 draft
KV cache	FP8
Benchmark cell	p512 / g128, C1
Median decode	84.65 tok/s
Mean decode	84.49 tok/s
Min / Max	83.73 / 84.78 tok/s
Median TTFT	0.3557 s
Reference card	83.7 tok/s

Conclusion: the published ~83.7 tok/s BF16-draft MTP4 B70 performance was successfully reproduced. The five measured runs were tightly clustered, indicating a stable result rather than a one-off.

2. Hardware and host

- Validation host: Dell XPS 8940 running Ubuntu 26.04 LTS.
- GPU: Intel Arc Pro B70 32 GB Battlemage, PCI address 04:00.0, Linux xe driver.
- B70 DRM nodes: /dev/dri/card2 and /dev/dri/renderD129.
- Integrated Intel UHD 750 remains present at /dev/dri/card1 and /dev/dri/renderD128.
- PCIe link was corrected and verified healthy at Gen3 x16.
- ZE_AFFINITY_MASK=0 selected the B70; vLLM reported approximately 30.3 GiB total device memory.
- Existing services were preserved: Ollama on host port 11434, llama.cpp SYCL on 11435, vLLM on 11436.

3. Baseline performance before vLLM

Stack	Model / quantization	Decode
Ollama Vulkan	Qwen3.8-27B Q4_K_M	11–12 tok/s
llama.cpp SYCL	Qwen3.8-27B Q4_K_M	27.4 tok/s
vLLM XPU MTP4	GPTQ INT4 + BF16 MTP	84.65 tok/s median

The working vLLM configuration was approximately 3.1× faster than llama.cpp SYCL and about 7.2× faster than Ollama Vulkan for decode.

4. Repository, image, model, and patches

Repository:
<https://github.com/SergiiB/intel-arc-pro-b70-inference-cookbook>

Local repository:

```
~/intel-arc-pro-b70-inference-cookbook
```

Pinned vLLM image:

```
vllm/vllm-openai-xpu@sha256:f01e24f6c7ff01f1e0662234255a1372297d1dbd89d003cf13c8fad3eab1ba4f
```

Model:

```
SergioB/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16
```

Model revision:

```
9d189a60e4c0ad7f9f47cd94bfa393ca10b3924e
```

Local model path:

```
$HOME/models/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16
```

Required patches:

```
patches/patch_mtp_nightly.py
```

```
SHA256 4d7a02c4ea10ca7c00dc89ad927fa3dafa747dbf0553d2adf24e30a3c53e9c14
```

```
patches/patch_mtp_boundary.py
```

```
SHA256 41d2f74e5fef1f074b76b5a90dd1016de437228431802cfb1fa7bd7ce4cc9b50
```

Do not re-download the model when the local directory already exists. The model is GPTQ INT4, group size 128, symmetric, with MTP tensors deliberately retained in BF16.

5. Docker device mapping and oneCCL/XCCL troubleshooting

Start with the normal Intel GPU device mapping. On the validation system, oneCCL/XCCL initially failed with errors involving `ze_fd_manager` and an inability to open the DRM device directory.

Default Docker device mapping:

```
--device /dev/dri:/dev/dri
-v /dev/dri:/dev/dri:ro
--group-add "$RENDER_GID"
```

If oneCCL/XCCL fails with `ze_fd_manager`, DRM device-directory, or similar errors, add:

```
--cap-add SYS_PTRACE
--security-opt seccomp=unconfined
--ipc=host
```

The three additional permissions above were required on the validation host and successfully resolved the oneCCL/XCCL startup failure. Treat them as a troubleshooting workaround rather than assuming they are required on every Intel Arc Pro B70 system. If the server initializes correctly without them, use the simpler device mapping.

6. Known-good server launch

Start with the standard `/dev/dri` mapping. This baseline uses 131072 context, GPU memory utilization 0.88, FP8 KV cache, max 64 sequences, 8192 batched tokens, prefix caching disabled, and four MTP speculative tokens. If oneCCL/XCCL startup fails, use the Section 5 workaround.

```
cd ~/intel-arc-pro-b70-inference-cookbook
```

```
export IMAGE='vllm/vllm-openai-xpu@sha256:f01e24f6c7ff01f1e0662234255a1372297d1dbd89d003cf13c8fad3eab1ba4f'
export MODEL_DIR="$HOME/models/Qwen3.8-27B-GPTQ-Int4-sym-G128-MTP-BF16"
export B70_RENDER_NODE="/dev/dri/renderD129"
export RENDER_GID="$(stat -c '%g' "$B70_RENDER_NODE")"
```

```
docker rm -f qw38speed 2>/dev/null || true
```

```
docker run -d \
  --name qw38speed \
  --restart unless-stopped \
  -p 11436:8000 \
  --device /dev/dri:/dev/dri \
  -v /dev/dri:/dev/dri:ro \
  --group-add "$RENDER_GID" \
  -v "$MODEL_DIR:/model:ro" \
```

```

-v "$PWD/patches/patch_mtp_nightly.py:/patch_mtp.py:ro" \
-v "$PWD/patches/patch_mtp_boundary.py:/patch_boundary.py:ro" \
-e VLLM_TARGET_DEVICE=xpu \
-e ZE_FLAT_DEVICE_HIERARCHY=COMPOSITE \
-e ZE_AFFINITY_MASK=0 \
-e B70_MTP_BF16_DRAFT=1 \
-e VLLM_XPU_ENABLE_XPU_GRAPH=1 \
-e PYTORCH_ALLOC_CONF=expandable_segments:True \
--entrypoint bash \
"$IMAGE" \
-lc 'set -e
python /patch_mtp.py
python /patch_boundary.py
exec vllm serve /model \
--quantization gptq \
--dtype float16 \
--max-model-len 131072 \
--gpu-memory-utilization 0.88 \
--kv-cache-dtype fp8 \
--port 8000 \
--max-num-seqs 64 \
--max-num-batched-tokens 8192 \
--no-enable-prefix-caching \
--served-model-name qwen38 \
--language-model-only \
--speculative-config "{\"method\":\"mtp\",\"num_speculative_tokens\":4}"'

```

7. Startup verification

```
docker logs -f qw38speed
```

```
# Healthy endpoint:
curl -fsS http://127.0.0.1:11436/health && echo "HEALTH OK"
```

```
# Confirm served model:
curl -fsS http://127.0.0.1:11436/v1/models | jq
```

Do not benchmark until the log contains: “Application startup complete.”

Useful successful-startup observations included: XPUwNa16LinearKernel for AutoGPTQ, Triton/FLA GDN paths, FlashAttention v2, successful XPU graph capture, and approximately 30.3 GiB detected B70 memory.

8. Cold-run behavior and MTP verification

The first inference after launch triggered Triton JIT compilation for EAGLE/MTP kernels. This caused a temporary latency spike and must not be treated as a benchmark result.

```

Typical first-run warnings:
Triton kernel JIT compilation during inference:
  eagle_prepare_next_token_padded_kernel
  eagle_step_slot_mapping_metadata_kernel
  eagle_prepare_inputs_padded_kernel

```

MTP was confirmed active by SpecDecoding metrics. Early runs showed mean acceptance length around 2.67–2.73 and average draft acceptance around 42–43%. The vLLM periodic logger showed 12.8 tok/s, but that was a reporting-window artifact and not the actual post-first-token decode rate.

9. Exact p512/g128 benchmark procedure

Use one same-shape warmup followed by five measured C1 requests. The client uses monotonic timing and computes $(\text{completion_tokens} - 1) / (\text{request_end} - \text{first_generated_token})$. It obtains actual token counts from the final OpenAI-compatible streaming usage block rather than counting SSE chunks.

```

python3 - <<'PY'
import json, time, urllib.request, statistics

BASE = "http://127.0.0.1:11436"

seed = (
    "A retrieval augmented generation system retrieves relevant source passages, "
    "constructs a grounded context, sends the evidence to a language model, and "
    "generates a concise answer based on the supplied information. "
)

text = seed
while True:
    req = urllib.request.Request(
        BASE + "/tokenize",
        data=json.dumps({"model": "qwen38", "prompt": text}).encode(),
        headers={"Content-Type": "application/json"},
    )
    with urllib.request.urlopen(req) as r:
        tok = json.load(r)
        if len(tok["tokens"]) >= 512:
            break
        text += seed

ids = tok["tokens"][:512]

req = urllib.request.Request(
    BASE + "/detokenize",
    data=json.dumps({"model": "qwen38", "tokens": ids}).encode(),
    headers={"Content-Type": "application/json"},
)
with urllib.request.urlopen(req) as r:
    text = json.load(r)["prompt"]

def run():
    payload = {
        "model": "qwen38",
        "prompt": text,
        "max_tokens": 128,
        "temperature": 0,
        "ignore_eos": True,
        "stream": True,
        "stream_options": {"include_usage": True}
    }

    req = urllib.request.Request(
        BASE + "/v1/completions",
        data=json.dumps(payload).encode(),
        headers={"Content-Type": "application/json"},
    )

    start = time.perf_counter()
    first = None
    end = None
    usage = None

    with urllib.request.urlopen(req, timeout=300) as r:
        for raw in r:
            line = raw.decode().strip()
            if not line.startswith("data: "):
                continue
            data = line[6:]
            if data == "[DONE]":
                end = time.perf_counter()
                break
            obj = json.loads(data)
            if obj.get("usage"):
                usage = obj["usage"]
            choices = obj.get("choices", [])
            if choices and choices[0].get("text") and first is None:
                first = time.perf_counter()

```

```

if end is None:
end = time.perf_counter()

ct = usage["completion_tokens"]
pt = usage["prompt_tokens"]
return {
    "prompt": pt,
    "completion": ct,
    "ttft": first - start,
    "post": end - first,
    "rate": (ct - 1) / (end - first)
}

print("Warmup...")
w = run()
print(f"warmup: {w['rate']:.2f} tok/s TTFT={w['ttft']:.4f}s")

rates = []
ttfts = []

for i in range(1, 6):
    r = run()
    rates.append(r["rate"])
    ttfts.append(r["ttft"])
    print(
        f"run {i}: {r['rate']:.2f} tok/s "
        f"TTFT={r['ttft']:.4f}s "
        f"prompt={r['prompt']} completion={r['completion']}"
    )

print()
print(f"MEDIAN DECODE: {statistics.median(rates):.2f} tok/s")
print(f"MEAN DECODE: {statistics.mean(rates):.2f} tok/s")
print(f"MIN/MAX: {min(rates):.2f} / {max(rates):.2f} tok/s")
print(f"MEDIAN TTFT: {statistics.median(ttfts):.4f} s")
PY

```

10. Reproduced benchmark output

```

Warmup...
warmup: 83.82 tok/s TTFT=1.1475s
run 1: 83.73 tok/s TTFT=0.3662s prompt=512 completion=128
run 2: 84.65 tok/s TTFT=0.3573s prompt=512 completion=128
run 3: 84.70 tok/s TTFT=0.3557s prompt=512 completion=128
run 4: 84.61 tok/s TTFT=0.3531s prompt=512 completion=128
run 5: 84.78 tok/s TTFT=0.3553s prompt=512 completion=128

MEDIAN DECODE: 84.65 tok/s
MEAN DECODE: 84.49 tok/s
MIN/MAX: 83.73 / 84.78 tok/s
MEDIAN TTFT: 0.3557 s

```

11. Important measurement lesson

Do not use SSE event count as token count. In one test, 128 output tokens arrived in only 27 SSE chunks. Counting chunks incorrectly produced 16.99 chunks/s even though the true post-first-token rate was about 84–85 tok/s. Likewise, vLLM's periodic 'Avg generation throughput' logger can average over its reporting interval and showed 12.8 tok/s for these short requests. For this benchmark, use the streaming usage token count and client-side first-token/end timestamps.

12. Troubleshooting checklist

- If XCCL/oneCCL fails with `ze_fd_manager` or 'could not open device directory', first verify the standard `/dev/dri` mapping. If the failure persists, add the `SYS_PTRACE`, `seccomp=unconfined`, and `--ipc=host` workaround from Section 5.
- If the wrong Intel GPU is selected, verify `ZE_AFFINITY_MASK=0` and confirm vLLM sees about 30.3 GiB, consistent with the B70.
- If the first request is slow, inspect logs for Triton JIT warnings; run the same shape once as warmup before measuring.
- If MTP appears inactive, inspect SpecDecoding metrics for accepted/drafted tokens and acceptance length.
- If a benchmark reports ~12.8 tok/s from the periodic vLLM logger, do not compare it to the cookbook post-first-token metric.
- If a streaming script reports very few 'tokens', check whether it is counting SSE chunks rather than the usage `completion_tokens` field.
- Preserve Ollama/OpenWebUI and llama.cpp by keeping vLLM on host port 11436.

13. Optional power-cap check

Before changing power limits, inspect the actual xe hwmon values. Do not assume a sysfs path.

```
for h in /sys/class/drm/card2/device/hwmon/hwmon*; do
    echo "=== $h ==="
    cat "$h/name" 2>/dev/null
    cat "$h/power1_cap" 2>/dev/null
    cat "$h/power1_cap_max" 2>/dev/null
done
```

The reference campaign used a 230 W configured cap. The B70 on this validation system had previously been observed around 228–230 W under load, so no blind power-cap change should be made.

14. RAG deployment notes

For retrieval-augmented generation (RAG), keep retrieval authoritative: retrieve relevant evidence first, keep the supplied context concise, instruct the model to answer from that evidence, and measure retrieval latency separately from model TTFT and decode throughput.

For application validation, benchmark representative evidence payloads and record retrieval latency, prompt tokens, TTFT, prefill throughput, decode throughput, total latency, correctness, and concurrency separately.

15. Preserve this known-good baseline

- Do not apply optional GDN mixed-split v5, draft-lmhead INT4, or draft-MTP INT4 patches until the baseline is archived.
- Keep the pinned image digest, model revision, patch hashes, Docker flags, environment variables, and vLLM arguments together.
- Record any future change as a separate benchmark configuration rather than overwriting this baseline.
- The current full cookbook matrix may evolve; this document preserves the configuration that actually reproduced the 83.7 tok/s BF16-draft reference on September 3, 2026.

16. Quick recovery sequence

1. `cd ~/intel-arc-pro-b70-inference-cookbook`
2. Identify and confirm your B70 card and render node; do not assume `card2/renderD129` on another host.
3. Confirm the model directory exists locally.
4. Launch `qw38speed` using the standard `/dev/dri` mapping in Section 6.
5. If startup fails with `ze_fd_manager`, DRM device-directory, or similar oneCCL/XCCL errors, add the Section 5

workaround.

6. Follow logs until "Application startup complete."
7. `curl http://127.0.0.1:11436/health`
8. Confirm `/v1/models` advertises `qwen38`.
9. Confirm the correct 32 GB B70 was selected.
10. Run one p512/g128 same-shape warmup.
11. Run five measured p512/g128 C1 requests.
12. Expect approximately 84-85 tok/s median decode and ~0.35 s warmed TTFT on the Dell B70 validation system.

17. Reference notes

Cookbook repository: [SergiiB/intel-arc-pro-b70-inference-cookbook](#). Relevant documentation includes `docs/qwen38-27/QWEN38-VLLM-XPU.md` and `docs/FULL-SETUP-COMMANDS.md`. This manual intentionally records the known-good configuration observed in this troubleshooting session so that later upstream changes do not erase the exact reproduction recipe.